

3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Before fork():\n");
    printf("PID = %d\n", getpid());
    printf("PPID = %d\n\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("CHILD PROCESS:\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("State = Running\n\n");

        sleep(3);

        printf("CHILD PROCESS AFTER SLEEP:\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("State = Running\n");

        exit(0);
    }
    else
    {
        // Parent process
        printf("PARENT PROCESS:\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("Child PID = %d\n", pid);
        printf("State = Running\n\n");

        printf("Parent waiting for child...\n");
        wait(NULL);

        printf("\nParent: Child process has terminated.\n");
        printf("Parent PID = %d\n", getpid());
        printf("State = Running\n");
    }

    return 0;
}
```

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Process started\n");
    printf("PID = %d\n", getpid());
    printf("PPID = %d\n", getppid());

    printf("Running for 5 seconds...\n");

    for (volatile long i = 0; i < 2000000000L; i++)
        ;

    printf("Now entering WAITING/SLEEPING state...\n");
    sleep(10);

    printf("Process completed. Terminating...\n");

    return 0;
}
```

```
gowtham@DESKTOP-2F03K3T:~/2520090202_osspractical.c$ mkdir practical-3.c
gowtham@DESKTOP-2F03K3T:~/2520090202_osspractical.c$ cd practical-3.c
gowtham@DESKTOP-2F03K3T:~/2520090202_osspractical.c/practical-3.c$ nano practical-3.c
gowtham@DESKTOP-2F03K3T:~/2520090202_osspractical.c/practical-3.c$ gcc practical-3.c -o
practical-3
gowtham@DESKTOP-2F03K3T:~/2520090202_osspractical.c/practical-3.c$ ./practical-3
```

Parent Process

Parent PID = 12456

Child PID = 12457

Child Process

Child PID = 12457

Parent PID = 12456

Child Process Completed

```
gowtham@DESKTOP-2F03K3T:~/2520090202_osspractical.c/practical-3.c$ cd ..
```

```
gowtham@DESKTOP-2F03K3T:~/2520090202_osspractical.c$ █
```